Relatório Técnico — Ressonance (Trabalho Individual II — SCOM)

Estado: relatório-base das correções executadas no código. Os dados e capturas que dependem de MySQL local, navegador e Git remoto ainda devem ser incluídos antes da submissão final. Data da revisão: 10/11/2026.

1. Introdução

O Ressonance é uma plataforma de catálogo e resenhas de álbuns musicais. O Trabalho Individual II acrescenta persistência relacional, autenticação e autorização a uma interface de múltiplas páginas iniciada no Trabalho I. O foco desta revisão foi diminuir riscos de segurança, corrigir falhas de validação, melhorar a arquitetura e documentar e testar a API.

2. Evolução em relação ao Trabalho I

A base visual HTML/CSS/JS foi preservada. O aplicativo utiliza backend em Express 5 com MySQL, bcrypt, JWT, Zod 4 e logs Winston. Nesta revisão o código foi modularizado em `app.js`, `repository.js`, `schemas.js`, `config.js` e `logger.js`, com autenticação revogável, rota PUT de álbuns e consulta agregada de médias.

3. Requisitos funcionais

Usuários podem cadastrar-se, autenticar-se, consultar catálogo e resenhas, publicar e apagar resenhas próprias, consultar e atualizar perfil e escolher até três gêneros favoritos. Administradores podem criar, atualizar e apagar álbuns, cadastrar faixas e moderar resenhas. Visitantes podem consultar os conteúdos públicos.

4. Arquitetura do sistema

Arquitetura em camadas: frontend (HTML/CSS/JS) → API REST Express (`app.js`) → validação Zod (`schemas.js`) → persistência parametrizada (`repository.js`) → MySQL. Infraestrutura isolada em `server.js`, `config.js`, `db.js` e `logger.js`. O repositório de dados é injetado na aplicação para facilitar testes. A matriz de permissões está em `docs/ARQUITETURA.md`.

5. Modelo de dados

O arquivo `schema.sql` define tabelas `utilizadores`, `albuns`, `faixas`, `resenhas` e `utilizadores_generos`, com PKs, FKs e `ON DELETE CASCADE` para entidades dependentes, índices relevantes, restrição de e-mail e número de faixa únicos. Há restrição de nota de 0 a 5. A coluna `token_version` permite revogação. Para base anterior, foi criada a migração `migrations/001_token_version.sql`. A comparação efetiva com `SHOW CREATE TABLE` do banco real ainda está pendente.

6. Estratégia de autenticação

Senha em hash bcrypt (fator 12) e token JWT HS256 assinado com `JWT_SECRET` obrigatório fora do código, expiração em 2 horas, emissor e audiência verificados. Cada requisição autenticada lê o usuário atual do banco e compara a versão de token. Logout incrementa `token_version`, invalidando tokens antigos. O bootstrap de administrador é comando local que exige senha fornecida por variável de ambiente (sem credencial padrão).

7. Estratégia de segurança

Consultas parametrizadas evitam concatenação SQL. Validação backend Zod 4 exige tipo, tamanho e formato, incluindo rejeição do campo `perfil` no cadastro. Saídas dinâmicas HTML são escapadas, e capas são filtradas por protocolo/URL segura; respostas de erro não contêm SQL nem stack trace

em produção. CORS limitado às origens configuradas. JWT Bearer não é enviado automaticamente pelo navegador, reduzindo a exposição a CSRF por cookie; o token em `localStorage` exige atenção permanente a XSS. Também são aplicados limites de corpo JSON.

8. Endpoints implementados

São 16 endpoints HTTP documentados individualmente, com métodos, URLs, parâmetros, exemplos, respostas e erros possíveis, em `docs/API.md`. Inclusões desta revisão: `POST /api/auth/logout`, `GET /api/albuns/:id` e `PUT /api/albuns/:id`. Os demais endpoints foram preservados na função.

9. Estratégia de testes

Usamos `node:test` e chamadas HTTP reais ao aplicativo Express com repositório simulado, separando regras de negócio do MySQL. O conjunto inclui casos válidos e inválidos: registro, login, permissões, manipulação de tokens, validação, consultas parametrizadas, escaping, logout e respostas de erro.

10. Resultados dos testes de segurança

20 testes automatizados passaram; 0 falharam na execução registrada em `docs/evidencias/testes-automatiz`
Cinco cenários recomendados para evidência adicional: SQL Injection, XSS, rota protegida sem login, perfil insuficiente e reutilização de token revogado (`docs/TESTES_E_SEGURANCA.md`). Os testes não substituem screenshots do navegador nem uma integração real com MySQL.

11. Logs e observabilidade

O logger Winston grava mensagens com timestamp para login válido, falha de autenticação, criação de entidades e erro interno; arquivo padrão `logs/app.log` ignorado pelo Git. Conteúdo técnico interno aparece nos logs; na resposta HTTP, só em modo `development`. Uma amostra de log do servidor em funcionamento deve ser capturada durante a apresentação.

12. Desempenho e limitações

A home antes solicitava `/api/albuns` e depois uma requisição por álbum para calcular notas. A revisão moveu a agregação de médias para uma consulta SQL e eliminou as requisições adicionais, reduzindo as chamadas ao catálogo para uma. **Não foram medidos milissegundos reais** porque MySQL e navegador operacional não estavam disponíveis para ensaio neste ambiente. Medições reais e capturas devem ser inseridas pelo responsável, assim como a validação de migração e testes end-to-end.

13. Uso de IA generativa

Uma ferramenta de IA generativa (ChatGPT da OpenAI) foi utilizada para inspecionar o código enviado, localizar vulnerabilidades, propor refatorações, implementar alterações, elaborar testes e produzir documentação. O resultado foi verificado com 20 testes automatizados executados. A avaliação crítica e execução no banco local, as capturas e a submissão final permanecem sob responsabilidade do autor do trabalho. Esta declaração deve ser revisada para refletir fielmente a contribuição realizada.

14. Conclusões

Os problemas críticos de escalada de privilégios e segredo JWT padrão foram removidos, assim como a senha fixa de criação de administrador. O logout passou a revogar tokens, a API ganhou UPDATE de álbuns e validação estruturada, a interface passou a escapar dados externos, ganhou um servidor estático com arquivos permitidos e os principais fluxos possuem testes. Restam verificações que dependem do ambiente local e da entrega acadêmica, listadas em `docs/CHECKLIST.md`.

Anexos e evidências

- Script: `Ressonance-backend/schema.sql` e migração em `migrations/`.
- Documentação completa: `docs/API.md`.
- Execução dos testes: `docs/evidencias/testes-automatizados.txt`.
- Cenários de segurança e capturas pendentes: `docs/TESTES_E_SEGURANCA.md`.
- Logging e amostra pendente: `Ressonance-backend/logs/app.log` (não versionar).
- Git e tags: obter `git log --oneline --graph` no repositório real.
- Checklist preenchido: `docs/CHECKLIST.md`.

Anexo A — Matriz resumida de permissões

Operação	Visitante	Usuário	Administrador
Consultar catálogo e resenhas	Sim	Sim	Sim
Criar resenha e consultar perfil	Não	Sim	Sim
Alterar perfil	Não	Sim	Sim
Excluir resenha própria	Não	Sim	Sim
Moderar resenhas de terceiros	Não	Não	Sim
Cadastrar, editar e excluir álbum	Não	Não	Sim
Cadastrar faixas	Não	Não	Sim
Criar administrador pela API	Não	Não	Não

Anexo B — Evidências automatizadas

A suíte de testes confirma a rejeição de `perfil: admin` no cadastro, hash de senha, validação Zod, rejeição de credenciais incorretas, autenticação obrigatória, controle de administrador, inclusão/edição de álbum, permissão do autor da resenha, revogação por logout, rejeição de IDs usados em SQL Injection, escape HTML e URL de imagem, SQL parametrizado e códigos de erro HTTP. O resultado foi **20 testes executados, 20 aprovados, 0 falhas**. Para reproduzir: `cd Ressonance-backend && npm test`.

Anexo C — Guia de comprovação pendente

Para completar a entrega acadêmica, executar o projeto com MySQL real; fotografar respostas de cinco cenários de segurança no Postman e no navegador; registrar amostra do `app.log` sem dados confidenciais; medir a home pelo DevTools; e anexar `git log --oneline --graph`, branch e tag criadas no repositório verdadeiro. O pacote de código fornecido omite `.env`, `app.log` e `.git` por segurança.